

AIQT CLI

Milestone 8 Build Specification v0.3

Agent Driver Protocol and Stdin Input

Build-planning document for implementing AIQT Milestone 8. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 8 Build Specification
Version	v0.3
Milestone	M8 - Agent Driver Protocol and Stdin Input
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1-M7 complete; RC1 guided workflow hardening merged at 45d7174cbdf7aa1c99ef3e73027cdc5ead3686ad; tag rc1-guided-workflow-hardening
Primary Agent Target	Claude Code, Codex, or equivalent coding agent
Implementation Status	Review candidate before build handoff
Document Status	v0.3 review candidate. Resolves v0.2 findings: ImportResultData remains additive and preserves delegatedAction; delegatedResultSummary is defined; sourcePath null behavior is clarified; section numbering is fixed.

Document status: v0.3 review candidate. This document resolves the v0.2 contract cleanup findings and is ready for final technical review before build handoff.

Contents

- 1. Source Alignment
- 2. v0.3 Change Summary
- 3. Milestone Objective
- 4. Commands in Scope
- 5. Out of Scope
- 6. Agent Driver Model
- 7. Planning Interview Model
- 8. Stdin Import Model
- 9. Prompt Template Updates
- 10. Full-Stack Planning Guidance
- 11. Behavioral Position in the Workflow
- 12. Canonical Mutation Rules
- 13. Runlog Rules
- 14. Error Handling and Exit Codes
- 15. Data and Schema Contracts
- 16. Repository Implementation Scope
- 17. Testing Scope
- 18. Definition of Done
- 19. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Driver Prompt
- Appendix C - Example Interview Prompt
- Appendix D - Example Stdin Commands

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6, AIQT Technical Architecture Specification v0.3, the completed milestone sequence M1 through M7, and the RC1 guided workflow hardening pass. It assumes main is at merge commit 45d7174cbdf7aa1c99ef3e73027cdc5ead3686ad and tagged rc1-guided-workflow-hardening. The repository baseline is 411/411 tests passing after RC1 hardening.

Milestone tags are literal repository tags, not title-derived names. Use these actual tags: m1-canonical-state-foundation, m2-project-context-capture, m3-work-graph-planning, m4-agent-handoff-packet, m5-checkpoint-engine, m6-review-export-engine, m7-guided-workflow, and rc1-guided-workflow-hardening.

```
state -> graph -> packet -> checkpoint -> review/export -> guided agent operation
not:
spec -> milestone file -> task file -> prompt file -> ad-hoc JSON file management
```

M8 does not replace the strict M1-M7 engine. It improves how an external coding agent operates AIQT from a rough human idea. JSON remains the deterministic validation contract, but temporary JSON files stop being the primary agent communication method.

2. v0.3 Change Summary

v0.3 is a contract-cleanup pass over v0.2. It preserves the M8 scope and fixes the final review findings before build handoff: ImportResultData now extends the M7 contract additively by preserving delegatedAction; delegatedResultSummary is explicitly copied from the delegated command result summary; sourcePath behavior is defined for file and stdin modes; and section numbering is sequential in the table of contents and body.

Finding	M8 implication
Engine and guided UX work	The empty-folder flow completed through init, prompt, import, next, checkpoint, review, and export.

AIQT Milestone 8 Build Specification v0.3

Full-stack planning remains unproven	CLI dogfoods did not test Next.js, Supabase, Clerk, Shadcn, Vercel, routing, auth, data modeling, forms, or deployment assumptions.
Agent operation is still too manual	The user or agent currently manages update.json, plan.json, and checkpoint.json files as the primary interaction path.
Planning needs stronger implementation shaping	Generic work units sequence work but do not force enough data model, route, UI, auth, and validation detail for full-stack apps.
AIQT should remain deterministic	The fix is not API integration or autonomous AI. The fix is better agent-driver instructions and stdin-based structured input.

3. Milestone Objective

Milestone 8 implements the Agent Driver Protocol and stdin input support. Its purpose is to let a coding agent operate AIQT from a rough user request while preserving AIQT as a local workflow state engine.

Target user command to an agent:

```
Build me a natural medicine marketplace using AIQT.
```

M8 must prove that AIQT can:

- generate a master agent-driver prompt that tells Claude Code or Codex how to operate AIQT safely;
- generate a planning interview from a rough idea so the agent asks useful questions before planning or coding;
- accept structured update, plan, and checkpoint input from stdin without requiring temporary JSON files;
- keep --from-file and .aiqt/inputs/ as optional human-review/debug paths;
- improve planning prompts for full-stack applications without schema redesign;
- preserve all M1-M7 and RC1 behavior and tests.

4. Commands in Scope

```
aiqt prompt driver
aiqt prompt driver --idea "<rough idea>"
aiqt prompt driver --json
aiqt prompt interview
aiqt prompt interview --idea "<rough idea>"
aiqt prompt interview --json
aiqt import update --stdin
aiqt import update --stdin --json
aiqt import plan --stdin
aiqt import plan --stdin --json
aiqt import checkpoint --stdin
aiqt import checkpoint --stdin --json
```

Prompt plan, update, and checkpoint templates are also in scope for wording updates that prefer stdin as the agent path and keep file input as the optional human-review path.

5. Out of Scope

- Claude, OpenAI, Codex, GitHub Copilot, or MCP API integration;
- automatic agent execution or background orchestration;
- automatic source-code modification by AIQT;
- web UI, hosted backend, or cloud synchronization;
- new canonical state files such as session.json, inputs.json, or agent.json;

AIQT CLI - Agent Driver Protocol and Stdin Input

- new markdown source-of-truth files such as AIQT.md, AGENTS.md, or CLAUDE.md;
- removing --from-file or .aiqt/inputs/;
- canonical schema redesign or AIQT_SCHEMA_VERSION bump unless unavoidable;
- replanning or changing M3 work graph mutation rules;
- packaging, npm publishing, or release documentation work.

6. Agent Driver Model

The agent driver model separates intelligence from workflow control. Claude Code or Codex remains the intelligence and operator. AIQT remains the state engine, validator, packet generator, and checkpoint controller.

Actor	Responsibility
Agent responsibility	Ask questions, synthesize structured inputs, run AIQT commands, implement only the current packet, report results.
AIQT responsibility	Persist canonical state, validate input contracts, compute workflow position, generate handoff packets, capture checkpoints, review/export.
Human responsibility	Provide product intent, answer clarification questions, approve or redirect scope, review outputs when needed.

6.1 aiqt prompt driver

aiqt prompt driver emits a master operating prompt for a coding agent. It must be usable before or after initialization where practical. If .aiqt/ is available, it should include current workflow state. If not, it should instruct the agent to initialize first.

- The prompt must explicitly tell the agent not to immediately implement from the rough idea.
- The prompt must instruct the agent to use AIQT as the workflow controller.
- The prompt must describe the sequence: init, start, interview, import update, prompt/import plan, next, implement, import checkpoint.
- The prompt must prefer stdin import for agent-generated JSON.
- The prompt must mention optional --from-file/.aiqt/inputs/ for human review/debugging.
- The prompt must warn against changing AIQT canonical files manually.

7. Planning Interview Model

aiqt prompt interview generates a project-discovery interview from a rough idea and any existing project context. It helps the agent ask questions before writing update or plan input. The interview prompt is not stored as canonical state.

Condition	Behavior
No idea supplied and no project objective	Generate a generic software-project interview and ask the agent to request the product idea first.
Idea supplied	Generate questions specific to the idea and likely application type.
Existing project context available	Avoid asking questions already answered in project.json unless clarification is needed.
Marketplace/web/full-stack implication detected	Include questions about users, auth, roles, data model, routes, forms, admin flows, payments, notifications, deployment, and non-goals.

7.1 Interview question categories

- Product objective and MVP slice;
- User roles and permissions;
- Core entities and data relationships;
- Authentication and onboarding model;
- Routes/pages and UI surfaces;

- Forms, validation, and error states;
- Business rules and non-goals;
- Integrations and external services;
- Technology stack and deployment assumptions;
- Validation strategy and acceptance criteria.

8. Stdin Import Model

M8 adds stdin as an input transport for existing import commands. It does not create a new mutation pathway. --stdin and --from-file must converge before validation and delegation.

Command	Behavior
aiqt import update --stdin	Read JSON from stdin, preflight parse it, then delegate to existing update behavior.
aiqt import plan --stdin	Read JSON from stdin, preflight parse it, then delegate to existing plan behavior.
aiqt import checkpoint --stdin	Read JSON from stdin, preflight parse it, then delegate to existing checkpoint behavior.

8.1 Stdin rules

- --stdin reads the entire standard input stream as UTF-8 text.
- Empty stdin is invalid input. Return exit code 3 with no mutation and recommend the relevant prompt command.
- Malformed JSON from stdin is invalid input. Return exit code 3 with no mutation.
- --stdin and --from-file are mutually exclusive. Combining them returns exit code 3.
- aiqt import <type> with neither --stdin nor --from-file returns status = needs_input, requiresHumanInput = true, exitCode = 10, nextRecommendedCommand = aiqt prompt <type>, and performs no mutation.
- If --stdin is supplied while process.stdin.isTTY is true, AIQT must not block indefinitely. Return status = failed, exitCode = 3, area = input, message = "--stdin requires piped input or redirected input.", nextRecommendedCommand = aiqt prompt <type>, and perform no mutation.
- When stdin parses as JSON, schema validation and workflow gates remain delegated to the existing update, plan, or checkpoint command/service behavior.
- No temporary file is written for stdin import.
- JSON mode must return the existing CommandResult shape with import metadata consistent with M7/RC1 behavior.

9. Prompt Template Updates

M8 updates prompt output so stdin is the preferred agent path and files are the optional human-review path.

Area	Required update
Preferred agent path	Pipe JSON directly into aiqt import <type> --stdin.
Optional human-review path	Save JSON under .aiqt/inputs/ and run aiqt import <type> --from-file <path>.
JSON contract	Prompts must still ask the agent to return valid JSON only when generating structured input.
No markdown fences	Prompts must continue warning the agent not to wrap JSON in markdown fences unless explicitly asked.
followUpCommand	PromptResultData.followUpCommand should prefer --stdin for agent mode where possible; CommandResult.nextRecommendedCommand should use followUpCommand when applicable.

10. Full-Stack Planning Guidance

M8 improves aiqt prompt plan so full-stack applications get implementation-shaped work units. This is a prompt/template improvement, not a required schema change.

When project context or idea includes signals such as Next.js, React, Supabase, Prisma, Clerk, Shadcn, Vercel, marketplace, booking, admin, auth, dashboard, or web app, the plan prompt must require the planning agent to consider:

- data model/schema and seed data assumptions;
- auth boundaries, roles, permissions, and protected routes;
- routes/pages in the Next.js App Router or equivalent router;
- UI components and forms;
- server actions, API routes, or backend service functions;
- form validation, error states, loading states, and empty states;
- environment variables and integration assumptions;
- test and validation strategy;
- deployment assumptions and non-goals.

The prompt must ask for work units that remain bounded. It must not ask the planning agent to create one massive work unit for the whole app.

11. Behavioral Position in the Workflow

M8 does not add a new canonical workflow stage. It is an agent-operation layer around the existing workflow.

```
init -> start/continue -> interview -> import update -> prompt/import plan -> next -> implement
-> import checkpoint -> review/export
```

State	Description
Before M8	Human often drives AIQT manually and manages JSON files under .aiqt/inputs/.
After M8	Agent can operate AIQT using driver/interview prompts and stdin import, while files remain optional.

12. Canonical Mutation Rules

- aiqt prompt driver is read-only and must not mutate canonical files.
- aiqt prompt interview is read-only and must not mutate canonical files.
- aiqt import <type> --stdin mutates exactly as the delegated update, plan, or checkpoint pathway mutates after validation succeeds.
- Stdin input must not be stored in .aiqt/inputs/ automatically.
- .aiqt/inputs/ remains optional and non-canonical.
- AIQT_SCHEMA_VERSION should not change unless implementation proves unavoidable.
- No new canonical files are introduced in M8.

13. Runlog Rules

- Prompt driver and prompt interview must not append runlog events because they are read-only prompt generation.
- M8 must not introduce a new import-specific runlog event.
- Successful stdin imports must append the same runlog events as the delegated update, plan, or checkpoint command.
- Failed prompt, driver, interview, stdin TTY, empty stdin, malformed stdin, or stdin/from-file conflict operations must not append runlog events.

- Import metadata may be returned in `CommandResult.data`, but runlog history remains owned by the delegated mutation command.

14. Error Handling and Exit Codes

Condition	Status	Exit code	Next recommendation
aiqt prompt driver unsupported option	failed	3	null or corrected prompt driver command
aiqt prompt interview unsupported option	failed	3	null or corrected prompt interview command
aiqt import <type> --stdin with empty stdin	failed	3	aiqt prompt <type>
Malformed JSON from stdin	failed	3	aiqt prompt <type>
--stdin and --from-file supplied together	failed	3	choose one input mode
Neither --stdin nor --from-file supplied	needs_input	10	aiqt prompt <type>
--stdin supplied while stdin is interactive TTY	failed	3	aiqt prompt <type>
Valid stdin JSON but schema invalid	failed	3	delegated command recommendation
Valid stdin JSON but workflow action blocked	blocked	2	delegated command recommendation, mapped to guided command where applicable
Successful stdin import mutation	passed	0	guided next recommendation

15. Data and Schema Contracts

M8 may add non-canonical result data for driver and interview prompt outputs. These are `CommandResult` data payloads only, not project/state schema changes. M8 must extend the existing M7 `ImportResultData` contract for stdin input. The extension is additive: `delegatedAction` remains unchanged from M7 and must not be renamed.

```
type PromptKind = "update" | "plan" | "checkpoint" | "driver" | "interview";
```

```
interface DriverPromptData {
  promptType: "driver";
  idea: string | null;
  prompt: string;
  preferredInputMode: "stdin";
  optionalInputMode: "from-file";
  nextRecommendedCommand: string;
}
```

```
interface InterviewPromptData {
  promptType: "interview";
  idea: string | null;
  detectedProjectType: string | null;
  questions: string[];
  prompt: string;
  followUpCommand: string;
}
```

```
interface ImportResultData {
  importType: "update" | "plan" | "checkpoint";
  delegatedAction: "update" | "plan" | "checkpoint";
  source: "file" | "stdin";
  sourcePath: string | null;
  followUpCommand: string | null;
}
```

AIQT Milestone 8 Build Specification v0.3

```
    delegatedResultSummary: string;
  }

  // M7 compatibility rule:
  // delegatedAction remains unchanged and must not be renamed to delegatedCommand.
  // M8 adds source, nullable sourcePath, and delegatedResultSummary.

  // sourcePath behavior:
  // --from-file mode: source = "file"; sourcePath = supplied path string.
  // --stdin mode: source = "stdin"; sourcePath = null.

  // delegatedResultSummary behavior:
  // delegatedResultSummary is copied from the delegated command result summary.
  // It must not invent a second summary or re-describe the mutation independently.
```

16. Repository Implementation Scope

Area	Expected change
src/cli/commands/prompt.command.ts	Add prompt driver and prompt interview subcommands.
src/cli/commands/import.command.ts	Add --stdin option and mutual-exclusion validation.
src/services/prompt-service.ts	Generate driver/interview prompts and update existing prompts to prefer stdin.
src/services/import-service.ts	Read stdin, preflight JSON, then delegate through existing import path.
src/templates/prompts/*.template.ts	Update prompt templates for stdin, agent-driver behavior, interview questions, and full-stack planning guidance.
src/core/input or filesystem helper	Add small stdin reader if no existing utility exists.
tests/unit and tests/integration	Add focused tests for commands, data payloads, stdin behavior, prompt text, and no mutation on failures.

17. Testing Scope

M8 must preserve the existing 411-test baseline and add focused tests for the new agent-driver and stdin behavior.

Test area	Required coverage
prompt driver	Outputs agent operating instructions; includes idea when supplied; prefers stdin path.
prompt interview	Outputs useful questions; marketplace/full-stack ideas trigger full-stack concerns.
stdin update	Valid JSON through stdin delegates to update and mutates the same way as file input.
stdin plan	Valid JSON through stdin delegates to plan and mutates the same way as file input.
stdin checkpoint	Valid JSON through stdin delegates to checkpoint and mutates the same way as file input.
stdin invalid JSON	Exit code 3, no mutation, clear input issue.
stdin empty	Exit code 3, no mutation, clear input issue.
stdin interactive TTY	Exit code 3, no hang, clear input issue.
neither input mode	Exit code 10, status needs_input, no mutation, recommends aiqt prompt <type>.
stdin + from-file	Exit code 3, no mutation.
prompt updates	Plan/update/checkpoint prompts show preferred stdin path and optional file path.
full-stack plan prompt	Mentions data model, auth, routes/pages, UI components, server actions/API, validation, deployment.
import result data	Stdin imports preserve delegatedAction, set source = stdin, sourcePath = null, and copy delegatedResultSummary from the delegated command result.

18. Definition of Done

- aiqt prompt driver works in human and JSON output modes;
- aiqt prompt interview works in human and JSON output modes;
- aiqt import update --stdin, aiqt import plan --stdin, and aiqt import checkpoint --stdin work and delegate correctly;
- --stdin and --from-file conflict is rejected with exit code 3;
- invalid, empty, or interactive-TTY stdin never mutates canonical files and never hangs;
- aiqt import <type> with neither input transport returns needs_input with exit code 10;
- ImportResultData preserves delegatedAction, reports source = stdin, sourcePath = null, and copies delegatedResultSummary from the delegated command result;
- existing --from-file behavior remains unchanged;
- prompt templates prefer stdin for agents and keep .aiqt/inputs/ as optional human-review workflow;
- full-stack planning guidance is present in prompt plan when relevant;
- all prior 411 tests still pass;
- pnpm typecheck, pnpm lint, pnpm test, pnpm build, and pnpm validate pass;
- docs/ remains ignored and uncommitted;
- no new canonical markdown files are created.

19. Agent Execution Packet

This section is the build handoff for Claude Code once M8 is approved.

Starting point:

- main after RC1 hardening merge commit 45d7174cbdf7aa1c99ef3e73027cdc5ead3686ad
- tag rc1-guided-workflow-hardening
- expected baseline: 411/411 tests passing

Branch:

feat/m8-agent-driver-stdin

Implement only M8 - Agent Driver Protocol and Stdin Input.

Required implementation:

1. Add aiqt prompt driver and aiqt prompt interview.
2. Add --stdin support to aiqt import update|plan|checkpoint.
3. Update prompt templates to prefer stdin as the agent path and keep --from-file as optional human-review path.
4. Improve full-stack planning guidance in aiqt prompt plan.
5. Preserve all M1-M7 and RC1 behavior.
6. Add focused tests for every M8 behavior.
7. Run pnpm typecheck, pnpm lint, pnpm test, pnpm build, and pnpm validate.

Out of scope:

- no AI/API integration
- no MCP server
- no autonomous execution
- no canonical schema redesign
- no packaging/publishing work
- no docs/ commits

Commit message:

feat: add agent driver and stdin import

Appendix A - Minimum Test Matrix

ID	Scenario	Expected exit
----	----------	---------------

AIQT CLI - Agent Driver Protocol and Stdin Input

AIQT Milestone 8 Build Specification v0.3

A1	aiqt prompt driver returns operating instructions	0
A2	aiqt prompt driver --idea includes the rough idea	0
A3	aiqt prompt driver --json returns DriverPromptData	0
A4	aiqt prompt interview --idea generates project-specific questions	0
A5	aiqt prompt interview detects marketplace/web/full-stack concerns	0
A6	aiqt import update --stdin accepts valid JSON and delegates to update behavior	0
A7	aiqt import plan --stdin accepts valid JSON and delegates to plan behavior	0
A8	aiqt import checkpoint --stdin accepts valid JSON and delegates to checkpoint behavior	0
A9	invalid stdin JSON returns exit code 3 and no mutation	3
A10	empty stdin returns exit code 3 and no mutation	3
A11	interactive TTY stdin returns exit code 3 and does not hang	3
A12	--stdin and --from-file together return exit code 3	3
A13	neither --stdin nor --from-file returns exit code 10 and needs_input	10
A14	prompt plan recommends --stdin as preferred path	0
A15	prompt plan still includes optional --from-file path	0
A16	full-stack plan prompt mentions schema/auth/routes/UI/API/forms/deployment/tests	0
A17	stdin plan example uses a schema-valid plan with one milestone and one work unit	0
A18	ImportResultData preserves delegatedAction and reports stdin sourcePath/delegatedResultSummary correctly	0
A19	all existing tests pass	0

Appendix B - Example Driver Prompt

You are using AIQT as the workflow controller for this project.

Do not immediately implement from the rough idea.

First use AIQT to capture context, ask questions, build a plan, and execute one bounded work unit at a time.

Preferred agent path:

- Run aiqt init if .aiqt/ does not exist.
- Run aiqt start.
- Run aiqt prompt interview --idea "Build me a natural medicine marketplace".
- Ask the user the generated questions.
- Convert answers into valid update JSON.
- Pipe the JSON into aiqt import update --stdin.
- Run aiqt prompt plan.
- Convert the plan into valid plan JSON.
- Pipe the JSON into aiqt import plan --stdin.

AIQT CLI - Agent Driver Protocol and Stdin Input

AIQT Milestone 8 Build Specification v0.3

- Run aiqt next.
- Implement only the current work unit.
- Pipe checkpoint JSON into aiqt import checkpoint --stdin.

Do not manually edit .aiqt/project.json or .aiqt/state.json.
Do not create markdown source-of-truth files.

Appendix C - Example Interview Prompt

Project idea: Build me a natural medicine marketplace.

Ask the user these questions before planning:

1. Who are the primary users: clients, professionals, admins, clinics, or all of these?
2. What is the MVP slice: discovery, professional profiles, booking requests, payments, or admin approval?
3. Should professionals self-register or be invited/admin-created?
4. Does admin need to approve professional profiles before public listing?
5. Are bookings online, in-person, or both?
6. Are payments included in MVP or handled outside the platform?
7. What information must a professional profile contain?
8. What services should professionals be able to offer?
9. What stack should be used: Next.js, Supabase, Clerk, Shadcn, Prisma, Vercel?
10. What are explicit non-goals for the MVP?
11. What validation should prove the first slice works?

After the user answers, produce AIQT update JSON and submit it through aiqt import update --stdin.

Appendix D - Example Stdin Commands

```
aiqt import update --stdin <<'JSON'
{
  "project": {
    "objective": "Build a natural medicine marketplace MVP.",
    "targetUsers": ["Client", "Professional", "Admin"],
    "preferredAgent": "claude-code",
    "existingRepositoryPath": "."
  },
  "context": {
    "constraints": ["Use Next.js App Router", "Use Supabase", "Use Clerk"],
    "nonGoals": ["Payments", "Reviews", "Subscriptions"],
    "technologyPreferences": ["Next.js", "TypeScript", "Shadcn UI", "Supabase", "Clerk",
"Vercel"],
    "businessRules": ["Professionals must be admin-approved before appearing publicly"],
    "architectureNotes": ["Keep the booking request flow simple for MVP"]
  },
  "requirements": []
}
JSON
```

```
aiqt import plan --stdin <<'JSON'
{
  "milestones": [
    {
      "clientKey": "foundation",
      "title": "Foundation",
      "objective": "Establish the first implementation slice."
    }
  ]
}
```

AIQT Milestone 8 Build Specification v0.3

```
],
  "workUnits": [
    {
      "clientKey": "initial-slice",
      "milestoneClientKey": "foundation",
      "title": "Implement initial slice",
      "objective": "Create the first bounded implementation unit.",
      "scope": ["Implement the first usable project slice"],
      "outOfScope": ["Do not implement unrelated future features"],
      "acceptanceCriteria": ["The first slice can be validated locally"],
      "validationCommands": ["pnpm validate"],
      "agentContextRefs": [],
      "suggestedFiles": []
    }
  ],
  "dependencies": []
}
JSON
```